

1. Memory Layout of a C++ Program

When a C++ program is compiled into an executable and loaded into RAM, it is divided into specific segments:

- **Text (Code) Segment:** Stores the compiled machine code (the actual instructions). It is read-only to prevent accidental modification.
- **Data Segment (Initialized):** Stores global and `static` variables that are explicitly initialized by the programmer (e.g., `int global_x = 10;`).
- **BSS Segment (Uninitialized Data):** Stores global and `static` variables that are uninitialized. The OS automatically zeroes these out (e.g., `int global_y;`).
- **Heap:** Used for dynamic memory allocation at run-time (using `new`). You must manage this manually (using `delete`).
- **Stack:** Stores local variables, function arguments, and execution context. It grows downwards and shrinks automatically as functions return (stack unwinding).

2. Local vs. Global Scope

- **Global Variables:** Declared outside any function. Accessible anywhere.
- **Local Variables:** Declared inside a block `{ }` . They die when the block ends.
- **The Shadowing Rule:** If a local variable has the same name as a global variable, the local one takes priority.
- **Scope Resolution Operator (`::`):** If a local variable shadows a global one, you can still access the global version by prefixing it with `::` (e.g., `cout << ::x;`).

3. The `const` Keyword (The Immutable Promise)

`const` is a promise to the compiler that a value will not change. It enables heavy compiler optimization (putting data in read-only memory).

A. `const` with Pointers (Read Carefully):

- `const int* ptr` (or `int const* ptr`): **Constant Data.** You can change where the pointer points, but you *cannot* change the value at that address.
- `int* const ptr`: **Constant Pointer.** You can change the value at the address, but you *cannot* make the pointer point anywhere else.
- `const int* const ptr`: **Strict.** Neither the pointer nor the data can change.

B. `const` Member Functions:

If a class method is marked `const` , it guarantees it will not modify any class variables.

C++

```
void display() const {  
    // this->id = 5; // ERROR: Cannot modify members inside a const function  
}
```

- **The `mutable` Backdoor:** If you have a `const` function but absolutely must change one specific variable (like a hit-counter for debugging), declare that variable with the `mutable` keyword.

4. Initialization List vs. Assignment

Inside constructors, you can assign variables in the body or use an initialization list.

- **Why Initialization Lists are better:** They initialize variables at the exact moment of memory allocation.
- **When they are MANDATORY:** You *must* use initialization lists for `const` variables and reference (`&`) variables, because they must be assigned a value the second they are born and cannot be reassigned later in the constructor body.

5. Macros (#define) vs. inline Functions

Both are used to save the overhead of a function call, but they work at different compilation stages.

- **Macros (#define PI 3.14):** Handled by the **Preprocessor** before the code is even compiled. It blindly does a find-and-replace of text. It is fast but dangerous because it lacks type-checking and debugging support.
- **inline Functions:** Handled by the **Compiler**. If you mark a small function as `inline`, the compiler replaces the function call with the actual function code. It eliminates the stack-frame overhead of jumping to a function while maintaining strict C++ type safety. *Note: If an inline function is too large, modern compilers will ignore the `inline` keyword to prevent code bloat.*

6. The static Keyword

`static` detaches a variable or method from an individual object and attaches it to the class itself.

- **Static Variables:** All objects of the class share the exact same copy in memory. If Object A changes it, Object B sees the change.
- **Static Methods:** They can be called directly using the class name (`Student::printCount();`). Because they do not belong to any specific object, **they have no** `this` **pointer** and can *only* access other static variables.

7. Shallow vs. Deep Copy (The Double-Free Danger)

When you do `Student B = A;`, the default C++ copy constructor does a **Shallow Copy**.

- **The Trap:** If `A` has a pointer allocating heap memory, a shallow copy just copies the memory address to `B`. Now, both objects point to the same heap location. When `A` dies, its destructor frees the memory. When `B` dies, its destructor tries to free the *exact same memory again*, causing a fatal crash (Double Free).
- **The Solution:** You must write a custom **Deep Copy** constructor that allocates entirely *new* heap memory for `B` and copies the raw values over.

8. Can a Constructor be Private?

Yes. If a constructor is `private`, the outside world (`main()`) cannot instantiate the class directly.

- **Why do this?** This is the backbone of the **Singleton Design Pattern** (ensuring only one instance of a class ever exists, like a database connection) or **Factory Patterns**, where a dedicated `public static` method is responsible for validating and creating the objects safely.

9. The friend Keyword

Encapsulation strictly locks down `private` members. However, a class can explicitly declare an outside function or another class as a `friend`.

- **Effect:** The friend gets full VIP access to all `private` and `protected` members.
- **Software Engineering Warning:** Use this sparingly. It pierces the armor of encapsulation, leading to tightly coupled, hard-to-maintain code.

10. The Deep Mechanics of virtual (V-Table & V-Ptr)

When you declare a function as `virtual` to achieve Run-Time Polymorphism, C++ does heavy lifting behind the scenes:

1. **V-Table (Virtual Table):** The compiler creates a static array of function pointers for the class. It maps the virtual function names to their actual implementations.
2. **V-Ptr (Virtual Pointer):** The compiler secretly adds a hidden pointer (`_vptr`) inside every object of that class. This pointer points to the V-Table. At run-time, the program follows the V-Ptr to the V-Table to execute the correct overridden function.

Crucial Interview Questions:

- **Can a Constructor be Virtual? No.** A virtual call requires the V-Table. The V-Ptr isn't fully set up until *after* the base constructor finishes executing. You cannot dynamically resolve an object type that hasn't finished

being born yet.

- **Can a Destructor be Virtual? YES, and it MUST BE if you use inheritance.** If you delete a Derived object using a Base class pointer, a non-virtual destructor will only destroy the Base part, leaving the Derived part in memory (a massive leak). A `virtual` destructor ensures the Derived destructor executes first, followed by the Base destructor.